



Amar el diagrama de Clases completo: con atributos y métodos necesarios(abstracción). Constructor con parámetros. Nombre Relaciones, y poner cardinalidad mínima y máxima. Materializar las relaciones con atributos (MÁXIMO DETALLE) Implementar su código equivalente en JAVA.

El Departamento de Obras Públicas de una gran ciudad ha decidido desarrollar un sistema de información en tiempo real, con objeto de seguimiento y reparación de baches, producto del desgaste y accidentes en vía pública.

Un **ciudadano** informa de varios baches, cada uno es denunciado por un ciudadano. *Se busca el ciudadano, verifica la intersección, y genera un informe de rotura.* Alternativamente, si ya existe un informe, le comunica la fecha del mismo.

De los diversos **baches** y roturas similares, interesa conocer un número de identificación, y se guarda con, su tamaño, su posición exacta, es decir se compone de una **intersección** de esquina (*calleA, calleB, y altura*), barrio, y situación de emergencia o prioridad de reparación, que va de 1 a 5 (menos grave), y estado "sin reparar".

Se elabora, además, el **informe de rotura** que guarda: codInf, fecha informe, el problema, código de urgencia, y vincula al bache, al ciudadano informante, y un pedido de reparación (*null*). Si al buscar el ciudadano no existe (la primera vez), se registra nombre y su mail, para avisar cuando este reparado.

Por cada informe de rotura, en orden prioritario, se elabora un **pedido de obra** que incluye: número de pedido de obra, la ubicación, tamaño, fecha creación, fecha reparación(*null*), observaciones. Por cada pedido de obra se define una **brigada de reparación** que se identifica con un número, un nombre (Alfa, Delta, Bravo, RicoChet), la cantidad de trabajadores de la brigada, equipamiento y materiales necesarios. El pedido de obra se le entrega al responsable de la brigada quien le agrega la fecha final de la obra. Cada brigada de reparación se conforma dinámicamente, con 3 trabajadores, de los que se registraron (id, nombre, puesto, y si está libre u ocupado), más otro trabajador que *dirige* a otros trabajadores y es el jefe de brigada.

Cuando se termina la reparación del bache, el jefe de brigada informa al sistema, que la reparación se ha cumplido cargando fecha efectiva de reparación, y observaciones en el pedido de obra. El sistema actualiza automáticamente el estado en el bache como "*reparado*", y libera a la brigada asignada al pedido. Se imprime el pedido de obra y se composición y se envía al mail registrado del ciudadano.

A medida que se reciben los daños reportados por los ciudadanos como consecuencia de los baches, el sistema almacena los datos del ciudadano y de los daños. El Departamento de Obras públicas consulta si el bache ya está denunciado en cuyo caso informa al ciudadano la fecha final de la obra y si no está denunciado genera la denuncia automáticamente. **Métodos de Listados:** *informesporCiudadano, informesSinPedidos, informeBrigadasOciosas, pedidosPorBrigada, bachesSinReparar, y trabajadoresPorBrigada.*

1) Realice el **diagrama de secuencia** para los casos en que:

- A- Un ciudadano frecuente, se dispone a realizar una denuncia de un bache, con lo cual introduce sus datos en *IU pantalla*, se busca y una vez encontrado, le pide el ingreso de la ubicación del bache entre otros detalles, verifica la intersección y carga un bache, y procede a elaborar un 'informe de rotura' que guarda asociando bache, y ciudadano informante. Se da la orden la *IU impresor*, una constancia y se le informa a la *IU pantalla*, los datos junto a una fecha estimada de reparación.
- B- Eventualmente el representante en mostrador, busca el informe de rotura según prioridad, y para el mismo se elabora un pedido de obra con los datos arriba mencionados, fecha *now*, fecha reparación calculada (*now.plus(7 días)*).
- C- Diariamente se buscan pedido de obra del día, se buscan brigadas libres, y se asigna una brigada de reparación, que usaran su equipamiento y materiales para repararlo. El pedido de obra se le entrega al responsable de la brigada + 3 trabajadores, y el jefe le agrega la fecha final de la obra.

2) **Testing. Crear la clase Ciudadano con String nombre y contraseña de tipo entero, además de sus constructores con y sin parámetros.** Los 4 métodos serán los siguientes:

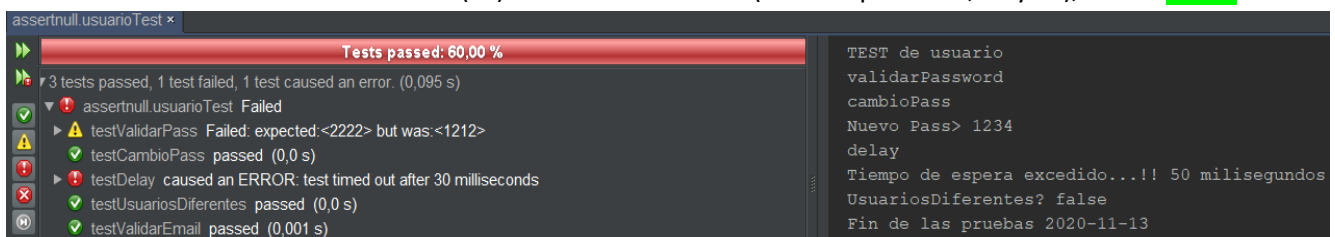
1. Crear la clase Junit desde NetBeans **ciudadanoTest**, que pone a prueba la clase anterior usando las anotaciones correspondientes. Cuenta con un atributo estatico *ciu*, de tipo ciudadano.
 2. Métodos **@Before** y **@After** imprime una línea separadora, y **@Before** además pone el número de test *i*.
 3. Use las anotación **@BeforeClass** en el método estático void *antesDeTodo()* para inicializar (*new*) el *ciu* con "*Anonimus*", 1212, luego mostrar mensaje de "*INGRESO de ciudadano del bien*".
 4. Use las anotación **@AfterClass** en el método estático void *despuesDeTodo()* que muestre *LocalDate.of* de la fecha de ayer, junto al msj "*Baches reparados, Ciudadanos Felices*".
- ✓ boolean *validarIngreso*(int psw) compara ambas contraseñas.
 - ✓ void *cambioPassword*(String newPass) convierte a entero newPass y lo asigna al actual



- ✓ void *delay*(int mili), pone un hilo a dormir Thread.sleep(mili), si el catch captura una InterruptedException, mostrara "Delay de {mili} milisegundos"
- ✓ boolean *ciudadanosDiferentes*(ciudadano c2), retorna el resultado de objetos iguales o diferentes

Métodos @Test dentro de **ciudadanoTest**

5. **testValidarUsr()**, define una contraseña '123456' con ella se invoca a validarPassword(contra). El resultado booleano, se comprara con AssertTrue, y debe **FALLAR**,
6. **testValidarEmail()** define un email = *******@*****.***. Si el assert del email.length() tiene un @ y un '.', y menos de 20 caracteres, nos dara True, esta prueba debe Tener **ÉXITO**.
7. **TestCambioPass()**, este @Test define un String newPass "1234"; Use usr.cambioPassword(newPass) y use assertEquals para comparar el valor entero 1234 con el actual pass de usr. Debe Tener **ÉXITO**, y mostrar el cambio.
8. @Test con timeout de 30, void **testDelay()** throws InterruptedException, llama a *delay()* en ciudadano, 45 milisegundos. Debe causar **ERROR** test timed out.
9. **testCiudadanosDiferentes()** crear 2 ciudadanos c1 y c2 que tengan valores diferentes de ciudadano y clave, llamar con c1.ciudadanosDiferentes(c2). Use assertNotSame("misma persona", c1 y c2), tendrá **ÉXITO**.



10. En otro archivo parameterCIUTest, cree una prueba parametrizada con el método **testValidarEmail()**. Use @RunWith(value=Parameterized.class) e Iterable tomarDatos previo @Parameters para la suma usando conjuntos de valores como los de la figura

"jorgesys@tototita.com"	true
"jorgesys@tototitacom"	false
"jorgesystototita.com"	false
"@jorgesystototita.com"	false
".jorgesystototita@com"	false

Devuelva con esos valores Arrays.asList(new Object[][] {{ x,x,x}{x,x,x}}). Los atributos estáticos, son string.

```
import javax.swing.JOptionPane;

public class Validar__mail2 {

    public static void main(String[] args) {

        boolean arroba = false;

        boolean punto = false;

        String mail = JOptionPane.showInputDialog("Ingrese su email por favor: ");

        for (int i = 0; i<mail.length(); i++) {

            if(mail.charAt(i)=='@' && mail.charAt(i)=='.') {

                arroba = true;
                punto = true;
            }

        }

        if (arroba == true && punto == true) {

            System.out.println("El mail ingresado es correcto");
        }
        else {
            System.out.println("El mail ingresado es incorrecto");
        }

    }

}
```

```
public static boolean ValidarMail(String email) {
    // Patron para validar el email
    Pattern pattern = Pattern.compile("^[_A-Za-z0-9-\\+]+(\\.\\[_A-Za-z0-9-\\+]+)*@[A-Za-z0-9-\\+]+(\\.\\[_A-Za-z0-9-\\+]+)*\\.\\[_A-Za-z]{2,}\\$");
    Matcher mather = pattern.matcher(email);return mather.find(); }

}
```